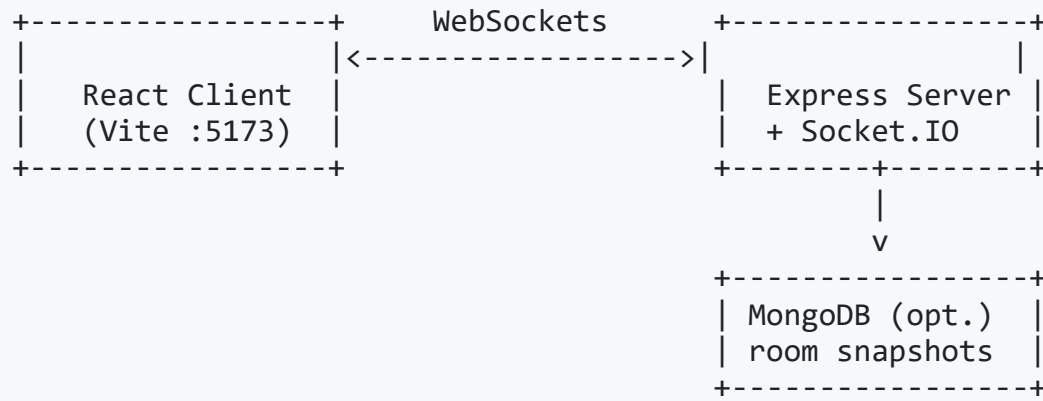


Liar's Deck

Design & Architecture

Server-authoritative multiplayer card game — WebSockets (Socket.IO), React + Vite client, Node/Express server.

High-level architecture



Principle: Rules, dealing, turn order, and **hidden information** are enforced on the server. Clients receive **partial** `gameState` broadcasts.

Requirement traceability

Six feature groups **F1–F6** with 18 requirements **R1.1–R6.3** (see `docs/requirements/requirements.md`):

- **F1** Lobby / matchmaking — **F2** Dealing — **F3** Turns
- **F4** Call Liar resolution — **F5** Hidden / partial views — **F6** Persistence & reconnect

Acceptance tests map to automated scenarios in `src/backend/tests/`.

Server authority & hidden state

- `hideGameState` : Broadcast `gameState` replaces each player's `hand` with placeholder cards (`rank: '?'`) so opponents' ranks are not leaked.
- `showCurrentPlayerHand` : On `getHand` , only the requesting socket receives a copy that restores **their** real hand for UI.
- `playCards` / `callLiar` : Validated against in-memory `gameState` in `app.ts` using `socket.data.player` and `whosTurn` .

Core entities (data model)

Entity	Role
Player	<code>playerId</code> , <code>name</code> , <code>ready</code> , <code>hand</code> , <code>revolver</code> , <code>isAlive</code>
GameState	<code>players</code> , <code>maxPlayers</code> , <code>gameStarted</code> , <code>whosTurn</code> , <code>declaredRank</code> , <code>moves</code>
LiarCard	<code>id</code> , <code>rank</code> (king / queen / ace / joker)
Lobby / room	Keyed by room name in <code>serverState.gameRooms</code>

Deck constants: `LIAR_DECK` , `TABLE_RANKS` , `REVOLVER_DECK` in `gameLogic.ts` .

Socket.IO events (selected)

Client → server: `createRoom`, `joinRoom`, `setUsername`, `leaveRoom`, `ready`, `getHand`, `playCards`, `callLiar`, `resetGame`

Server → client: `gameState`, `getHand`, `sessionToken`, `roomCreated`, `roomJoined`, `usernameSet`, `playCards`, `callLiar`, `gameEnd`, `error`, ...

Auth: `handshake.auth.sessionToken` for reconnection / session restore.

Persistence (optional)

`mongoPersistence.ts` : When `MONGODB_URI` is set, full room `GameState` documents are upserted by room name. On startup, `loadRoomSnapshots` hydrates `serverState.gameRooms` .

Socket.IO `connectionStateRecovery` mitigates brief drops without a full page reload.

Security notes (course scope)

- Do not trust the client for turn legality, card ownership, or “call liar” eligibility — all checked server-side.
- Production hardening would add TLS, authz per room, rate limits, and audited logging; this project demonstrates the **authoritative** pattern for class.

Source layout

Path	Purpose
<code>src/backend/app.ts</code>	HTTP + Socket.IO handlers
<code>src/backend/gameLogic.ts</code>	Rules, dealing, hide/show state
<code>src/frontend/src/App.tsx</code>	UI, socket wiring, animations
<code>shared/types</code>	Shared TypeScript types

Repository: https://github.com/joseph-ampfer/Bluff_Multiplayer